

Day 4-5 – Assignment C#.NET, OOPS

Parking Lot - Requirement 1

Let's start off by creating two **Vehicle** objects and check whether they are equal.

1) Create a **Vehicle** Class with the following private member attributes/variables:

Member Field Name	Type
_registrationNo	string
_name	string
_type	string
_weight	double
_ticket	Ticket

2) Create a **Ticket** Class with the following private member attributes/variables:

Member Field Name	Type
_ticketNo	string
_parkedTime	DateTime
_cost	double

3) Create / Generate appropriate Properties.

4) Add a parameterized constructor to take in all attributes in the given order

```
public Vehicle(string _registrationNo, string _name, string _type, double _weight, Ticket _ticket)
```

```
public Ticket(string _ticketNo, DateTime _parkedTime, double _cost)
```

5) When the "Vehicle" object is printed, it should display the following details: **[Override the ToString method]**

Print format:

Registration No: "_registrationNo"

Name: "_name"

Type: "_type"

Weight: "_weight"

Ticket No: "_ticketNo"

6) Two Vehicles are considered **same** if they have the **same _registrationNo, and _name**. Implement the logic in the appropriate function. (Case – insensitive) **[Override the Equals**

method]

7) The input format consists of Vehicle details separated by comma in the below order,

(registrationNo, name, type, weight,ticketNo,parkedTime,cost)

8) The Input to your program would be details of two Vehicles, you need to display their details as given in "5th point(refer above)" and compare the two Vehicles and display if the Vehicles are same or different.

Note: There is an empty line between display statements. Print the empty lines in Main function.

Display one digit after the decimal point for the double datatype.

9)Create a class **Program** with **Main** method to implement the above functionalities.

Sample Input & Output 1:

Enter Vehicle 1 details:

KL 75 F 9562,Pulsar,TwoWheeler,200,A1-032,12-05-2018 14:25:23,50

Enter Vehicle 2 details:

KL 75 F 9562,Pulsar,TwoWheeler,200,A1-032,12-05-2018 14:25:23,50

Vehicle 1

Registration No:KL 75 F 9562

Name:Pulsar

Type:TwoWheeler

Weight:200.0

Ticket No:A1-032

Vehicle 2

Registration No:KL 75 F 9562

Name:Pulsar

Type:TwoWheeler

Weight:200.0

Ticket No:A1-032

Vehicle 1 is same as Vehicle 2

Sample Input & Output 2:

Enter Vehicle 1 details:

AP 19 QS 4556,R3,TwoWheeler,196,A1-002,10-05-2018 11:05:21,100

Enter Vehicle 2 details:

MH 23 F 7856,Gixer,TwoWheeler,221,A1-013,11-05-2018 05:21:40,75

Vehicle 1

Registration No:AP 19 QS 4556

Name:R3

Type:TwoWheeler

Weight:196.0

Ticket No:A1-002





Microsoft DotNet

Free, Open-Source, Cross-Platform Framework



Vehicle 2

Registration No:MH 23 F 7856

Name:Gixer

Type:TwoWheeler

Weight:221.0

Ticket No:A1-013

Vehicle 1 and Vehicle 2 are different

Microsoft .NET by Ranjan B.

Parking Lot - Requirement 2

Now we are gonna start creating a **ParkingLot** and add **Vehicles** to it. Start with creating a **ParkingLot** and use menu-driven approach to add, remove, display details of the Vehicles in the ParkingLot.

a) Create a Class **Vehicle** with the following attributes:

Member Field Name	Type
_registrationNo	string
_name	string
_type	string
_weight	Double
_ticket	Ticket

b) Create a Class **Ticket** with the following attributes:

Member Field Name	Type
_ticketNo	string
_parkedTime	DateTime
_cost	Double

Mark all the attributes as private, Create / Generate appropriate Getters & Setters, Add a default constructor and a parameterized constructor to take in all attributes in the given order:

```
Vehicle( string _registrationNo, string _name, string _type, Double _weight,
Ticket _ticket )
Ticket(string _ticketNo, Date _parkedTime, Double _cost)
```

c) Create a Class **ParkingLot** with the following attributes:

Member Field Name	Type
_name	string
_vehicleList	List<Vehicle>

Mark all the attributes as private, Create / Generate appropriate Getters & Setters, Add a default constructor and a parameterized constructor to take in all attributes in the given order: **ParkingLot(string _name, List<Vehicle> _vehicleList)**. In constructor pass the vehicleList value as an empty list. Only one ParkingLot will be present at a time.

d) Create the following static method in **Vehicle** class,

Method Name	Description
public static Vehicle CreateVehicle(string detail)	This method accepts a string which contains vehicle details separated by commas. Split the details and create a vehicle object from the details and return it.

Free, Open-Source, Cross-Platform Framework

The Vehicle and Ticket details should be given as a comma-separated value in the below order,

_registrationNo, _name, _type, _weight, _ticketNo, _parkedTime, _cost

e) Create the following methods in `ParkingLot` class,

Method Name	Description
<code>public void AddVehicleToParkingLot(Vehicle vehicle)</code>	This method accepts a vehicle object and adds the vehicle to the vehicle list of the current <code>ParkingLot</code> .
<code>public bool RemoveVehicleFromParkingLot(string registrationNo)</code>	This method will get the registrationNo of the vehicle and delete the vehicle with the specified registrationNo from the current <code>ParkingLot</code> . If a vehicle with the given registrationNo found, delete the vehicle and return true . If a vehicle with the registrationNo is not found return false .
<code>public void DisplayVehicles()</code>	This method will display the vehicle list in the current <code>ParkingLot</code> . If the vehicle list is empty display "No vehicles to show" , else display "Vehicles in [parkingLot name]" and display all the vehicle details in the specified format. Where [parkingLot name] specifies the name of the parkingLot.

After deletion, if true is returned print **"Vehicle successfully deleted"**, else print **"Vehicle not found in parkinglot"**.

Note: The above print statements should be present in the main method.

Use : `DateTime.ParseExact(date, "dd-MM-yyyy HH:mm:ss", null)`

Print format:

Console.Write("{0,-15} {1,-10} {2,-12} {3,-7} {4}\n", "Registration No", "Name", "Type", "Weight", "Ticket No")

Display 1 digit after decimal point in Double.

Sample Input and Output:

Enter the name of the Parking Lot:

A1

- 1.Add Vehicle
- 2.Delete Vehicle
- 3.Display Vehicles

4.Exit

Enter your choice:

1

TN 56 AS 5489,R15,TwoWheeler,150,A1-001,11-05-2018 15:05:23,50

Vehicle successfully added

1.Add Vehicle

2.Delete Vehicle

3.Display Vehicles

4.Exit

Enter your choice:

3

Vehicles in A1

Registration No	Name	Type	Weight	Ticket No
TN 56 AS 5489	R15	TwoWheeler	150.0	A1-001

1.Add Vehicle

2.Delete Vehicle

3.Display Vehicles

4.Exit

Enter your choice:

2

Enter the registration number of the vehicle to be deleted:

TN 56 AS 5489

Vehicle successfully deleted

1.Add Vehicle

2.Delete Vehicle

3.Display Vehicles

4.Exit

Enter your choice:

2

Enter the registration number of the vehicle to be deleted:

TN 56 AS 5489

Vehicle not found in parkinglot

1.Add Vehicle

2.Delete Vehicle

3.Display Vehicles

4.Exit

Enter your choice:

3

No vehicles to show

1.Add Vehicle

2.Delete Vehicle

3.Display Vehicles

4.Exit

Enter your choice:

4

Parking Lot - Requirement 3

In this requirement, you need to validate the registrationNo of the Vehicle.

a) Create a class **Program** with the following static methods:

Method Name	Description
static bool ValidateRegistrationNo(string registrationNo)	Validate the registrationNo based on the rules given below. Return true if registrationNo is valid else return false

b) While validating a **registrationNo** follow the below rules.

1. The first 2 UPPERCASE letters indicate the state or Union Territory to which the vehicle is registered.
2. The second part contains 1-2 digit numbers which are the sequential number of a district.
3. Third part contains 0-2 uppercase letters.
4. The fourth part contains 1-4 digit number which is unique to each number plate.
5. Each part is separated by space.

Valid Format: **TS 01 K 1562**

Note: Print "**Registration No. is valid**" if registrationNo is valid else print "**Registration No. is invalid**".

All the above print statements are present in the main method.

[All text in bold corresponds to input]

Sample Input and Output 1:

Enter the registration no. to be validated:

TS 02 AB 9651

Registration No. is valid

Sample Input and Output 2:

Enter the registration no. to be validated:

UP 458 H 5462

Registration No. is invalid

Sample Input and Output 3:

Enter the registration no. to be validated:

TS 02 9651

Registration No. is valid

Parking Lot - Requirement 4

In this requirement develop a feature in which you can search a List of Vehicles by type, and parkedTime.

a) Create a Class **Vehicle** with the following attributes:

Member Field Name	Type
_registrationNo	string
_name	string
_type	string
_weight	Double
_ticket	Ticket

Mark all the attributes as private.

Create / Generate appropriate properties.

Add a default constructor and a parameterized constructor to take in all attributes in the given order:

Vehicle(string _registrationNo, string _name, string _type, Double _weight ,Ticket _ticket)

Create the following static method in **Vehicle** class,

Method Name	Description
public static Vehicle CreateVehicle(string detail)	This method accepts a string which contains vehicle details separated by commas. Split the details and create a vehicle object from the details and return it.

The Vehicle and Ticket details should be given as a comma-separated value in the below order,

_registrationNo, _name, _type, _weight, _ticketNo, _parkedTime, _cost

b) Create a Class **Ticket** with the following attributes:

Member Field Name	Type
_ticketNo	string
_parkedTime	DateTime
_cost	Double

Mark all the attributes as private.

Create / Generate appropriate properties,

Add a default constructor and a parameterized constructor to take in all attributes in the given order:

Ticket(String _ticketNo, Date _parkedTime, Double _cost)

c) Create a class **VehicleBO** with the following methods,

Method Name	Description
<pre>public List<Vehicle> FindVehicle(List<Vehicle> vehicleList, string type)</pre>	This method accepts a list of vehicles and type as arguments and returns a list of vehicles that matches with the given type.
<pre>public List<Vehicle> FindVehicle(List<Vehicle> vehicleList, DateTime parkedTime)</pre>	This method accepts a list of vehicles and parkedTime as arguments and returns a list of vehicles that matches with the given parkedTime.

The Vehicle and Ticket details should be given as a comma-separated value in the below order,
registrationNo, name, type, weight,ticketNo,parkedTime,cost

When the "vehicle" object is printed, it should display the following details

Print format:

Console.Write("{0,-15} {1,-10} {2,-12} {3,-7} {4}\n","Registration No","Name","Type","Weight","Ticket No")

Note: The vehicle lists are displayed in the main method.

If any other choice is selected, display **"Invalid Choice"**

If the search detail is not found, display **"No such vehicle is present"**

Display one digit after the decimal point for Double Datatype.

Use DateTime.ParseExact(date,"dd-MM-yyyy HH:mm:ss",null)

Sample test case 1:

Enter the number of vehicles:

6

AP 19 QS 4556,R3,TwoWheeler,196,A1-002,10-05-2018 11:05:21,100

MP 01 LK 0001,Hornet,TwoWheeler,163,A1-009,09-05-2018 08:13:24,50

Free, Open-Source, Cross-Platform Framework

MH 23 F 7856,Gixer,TwoWheeler,221,A1-013,11-05-2018 05:21:40,75
 GA 45 RF 9515,Duke,TwoWheeler,240,A1-024,10-05-2018 05:16:26,75
 GJ 83 AX 0545,Ciaz,FourWheeler,530,B4-030,10-05-2018 08:25:33,150
 HR 46 S 4523,Swift,FourWheeler,846,B4-021,09-05-2018 07:14:13,200

Enter a search type:

- 1.By type
- 2.By parked time

1

Enter the vehicle type

FourWheeler

Registration No	Name	Type	Weight	Ticket No
GJ 83 AX 0545	Ciaz	FourWheeler	530.0	B4-030
HR 46 S 4523	Swift	FourWheeler	846.0	B4-021

Sample test case 2:

Enter the number of vehicles:

6

AP 19 QS 4556,R3,TwoWheeler,196,A1-002,10-05-2018 11:05:21,100
 MP 01 LK 0001,Hornet,TwoWheeler,163,A1-009,09-05-2018 08:13:24,50
 MH 23 F 7856,Gixer,TwoWheeler,221,A1-013,11-05-2018 05:21:40,75
 GA 45 RF 9515,Duke,TwoWheeler,240,A1-024,11-05-2018 05:21:40,75
 GJ 83 AX 0545,Ciaz,FourWheeler,530,B4-030,10-05-2018 08:25:33,150
 HR 46 S 4523,Swift,FourWheeler,846,B4-021,09-05-2018 07:14:13,200

Enter a search type:

- 1.By type
- 2.By parked time

2

Enter the parked time:

11-05-2018 05:21:40

Registration No	Name	Type	Weight	Ticket No
MH 23 F 7856	Gixer	TwoWheeler	221.0	A1-013
GA 45 RF 9515	Duke	TwoWheeler	240.0	A1-024

Parking Lot - Requirement 5

In this requirement, you need to sort the list of vehicles based on `_weight` and `_parkedTime`.

a) Create a Class Vehicle with the following private member attributes/variables:

Member Field Name	Type
<code>_registrationNo</code>	string
<code>_name</code>	string
<code>_type</code>	string
<code>_weight</code>	double
<code>_ticket</code>	Ticket

Create / Generate appropriate properties.

Add a parameterized constructor to take in all attributes in the given order:

```
public Vehicle(string _registrationNo, string _name, string _type, double
_weight, Ticket _ticket)
```

b) Create the following static methods in the Vehicle class,

Method Name	Description
static Vehicle CreateVehicle(string detail)	This method accepts a String. The vehicle detail separated by commas is passed as the argument. Split the details and create a vehicle object and returns it.

c) Create a Class Ticket with the following attributes:

Member Field Name	Type
<code>_ticketNo</code>	string
<code>_parkedTime</code>	DateTime
<code>_cost</code>	double

Create / Generate appropriate Getters & Setters.

Add a parameterized constructor to take in all attributes in the given order:

```
public Ticket(string _ticketNo, DateTime _parkedTime, double _cost)
```

The vehicle details should be given as a comma-separated value in the below order, (`_registrationNo`, `_name`, `_type`, `_weight`, `_ticketNo`, `_parkedTime`, `_cost`)

d) The Vehicle class should implement the **Comparable** interface which sorts the Vehicle list based on weight. While comparing, all the weight attributes in the list are unique.

e) Create a class **parkedTimeComparer** which implements **IComparer** interface and sort the Vehicle list based on parkingTime. While comparing, all the parkingTime attributes in the

list are unique.

Get the number of Vehicles and vehicle details and create a vehicle list. Sort the Vehicles according to the given option and display the list.

When the "vehicle" object is printed, it should display the following details

Print format:

```
Console.WriteLine("{0,-15}{1,-10}{2,-12}{3,-7}{4}", this.RegistrationNo, this.Name,
this.Type, this.Weight, this.Ticket.TicketNo);
```

Display one digit after decimal point for Double datatype.

Sample test case 1:

Enter the number of the vehicles:

5

```
WB 91 F 1649,Vento,FourWheeler,530,B4-035,10-05-2018 20:45:27,300
PN 67 FK 0065,Polo,FourWheeler,543,B4-041,11-05-2018 21:35:56,150
GA 45 RF 9515,Duke,TwoWheeler,240,A1-024,10-05-2018 05:16:26,75
CH 51 P 9636,Ninja,TwoWheeler,287,A1-015,12-05-2018 15:05:30,100
KL 75 F 9562,Pulsar,TwoWheeler,200,A1-032,12-05-2018 14:25:23,50
```

Enter a type to sort:

- 1.Sort by weight
- 2.Sort by parked time

1

Registration No	Name	Type	Weight	Ticket No
KL 75 F 9562	Pulsar	TwoWheeler	200.0	A1-032
GA 45 RF 9515	Duke	TwoWheeler	240.0	A1-024
CH 51 P 9636	Ninja	TwoWheeler	287.0	A1-015
WB 91 F 1649	Vento	FourWheeler	530.0	B4-035
PN 67 FK 0065	Polo	FourWheeler	543.0	B4-041

Sample test case 2:

Enter the number of the vehicles:

5

```
MH 23 F 7856,Gixer,TwoWheeler,221,A1-013,11-05-2018 05:21:40,75
GA 45 RF 9515,Duke,TwoWheeler,240,A1-024,10-05-2018 05:16:26,75
WB 91 F 1649,Vento,FourWheeler,530,B4-035,10-05-2018 20:45:27,300
PN 67 FK 0065,Polo,FourWheeler,543,B4-041,11-05-2018 21:35:56,150
GJ 83 AX 0545,Ciaz,FourWheeler,530,B4-030,10-05-2018 08:25:33,150
```

Enter a type to sort:

- 1.Sort by weight
- 2.Sort by parked time

2

Registration No	Name	Type	Weight	Ticket No
GA 45 RF 9515	Duke	TwoWheeler	240.0	A1-024
GJ 83 AX 0545	Ciaz	FourWheeler	530.0	B4-030
WB 91 F 1649	Vento	FourWheeler	530.0	B4-035
MH 23 F 7856	Gixer	TwoWheeler	221.0	A1-013
PN 67 FK 0065	Polo	FourWheeler	543.0	B4-041

Parking Lot - Requirement 6

In this requirement, given a list of vehicles, you need to find type wise count of vehicles.

a) Create a Class **Vehicle** with the following attributes:

Member Field Name	Type
_registrationNo	string
_name	string
_type	string
_weight	double

Mark all the attributes as private.

Create / Generate appropriate properties,

Add a default constructor and a parameterized constructor to take in all attributes in the given order:

Vehicle(string _registrationNo, string _name, string _type, double _weight)

b) Create the following static method in Vehicle class,

Method name	Description
public static Vehicle CreateVehicle(string detail)	This method accepts a string which contains vehicle details separated by commas. Split the details and create a vehicle object from the details and return it.
public static SortedDictionary<string,int> TypeWiseCount(List<Vehicle>vehicleList)	This method accepts a list of vehicles and returns a SortedDictionary with typewise count of vehicles.

The Vehicle details should be given as a comma-separated value in the below order, **_registrationNo, _name, _type, _weight**

Print format:

Console.WriteLine("{0,-15} {1}\n", "Type", "No. of Vehicles")

Sample Input and Output 1:

Enter the number of vehicles

6

MP 01 LK 0001,Hornet,TwoWheeler,163

GA 45 RF 9515 ,Duke ,Two Wheeler,240

CH 51 P 9636 ,Ninja ,Two Wheeler,287

UP 12 FV 4681,Rapid,FourWheeler,684

WB 91 F 1649,Vento,FourWheeler,530

PN 67 FK 0065,Polo,FourWheeler,543

Type	No. of Vehicles
------	-----------------

FourWheeler	3
-------------	---

TwoWheeler	3
------------	---

☞*☞ Debug your doubts, deploy your success ☞*☞

Microsoft .NET by Ranjan B.